



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms-

Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- <u>Backtracking: N-Queens problem</u> , Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
 - Matrix chain Multiplication
 - Assembly line Scheduling
 - Longest Common Subsequence
 - 0/1 Knapsack Problem
 - TSP- Travelling Salesman Problem
- **Backtracking**
 - **N-queens Problem**
 - Sum of Subsets
 - Graph Coloring
- **Branch and Bound**
 - LIFO and FIFO Methods
 - Job Selection Problem
 - 0/1 Knapsack Problem



CBS3003- Design and Analysis of Algorithms- References

Text Books:

1. Horowitz, E., Sahni, S., & Rajasekaran, S. Fundamental of computer algorithms, Hyderabad, Universities Press; Second edition, 2008.
2. Kleinberg J, Tardos E. Algorithm design. Pearson Education India; 2006.

Reference Books:

1. Knuth Donald E, “ Art of Computer Programming: Fundamental algorithms”, Third edition, Pearson Publishers, 2011.
2. Pat Morin, “ Open Data Structures: An Introduction(Open Paths to Enriched Learning)”, 31 st Edition, UBC Press, 2013.



BACKTRACKING

- One of the mostly used general design technique which follows a strategy for guessing at a solution and backing up when deadlock situation is reached.
- Many problems which deal with **searching for a set of solutions** or **which ask for an optimal solution satisfying some constraints** can be solved using the backtracking.
- In many applications, the desired solution is expressible as an n-tuple $(x_1, \dots, x_n)$, where the x_i are chosen from some finite set S_i .
- Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$



BACKTRACKING

- **Explicit and Implicit Constraints**

- Explicit constraints are rules that restrict each X_i to take on values only from a given set.
- Explicit constraints depend on the particular instance “ I ” of the problem being solved. All tuples that satisfy the explicit constraints define a possible solution space for “ I ”.
- Implicit constraints are rules that determine which of the tuples in the solution space of “ I ” satisfy the criterion function.



BACKTRACKING

- **N-Queens Problem**

- Place “n” queens on a $n \times n$ chessboard so that no two attack, that is, so that no two of them are on the same row, column, or diagonal. For Example : 8×8
- All solutions to the 8-queens problem can therefore be represented as **8-tuples $(x_1, \dots, x_8)$ where x_i is the column on which queen i is placed.**



BACKTRACKING

- **N-Queens Problem**

- Explicit constraints using this formulation are $S_i = \{1, 2, 3, 4, 5, 6, 7, 8\}$, Solution space consists of **$64C_8$ ways = 4,426,165,368 ways**
- **Any observation to eliminate many arrangements??**
 - No queen can reside in a row or a column that contains another queen- **40,320 (8!) arrangements of queens and also to be checked for attacks along diagonals.**
- The implicit constraints for this problem are that no two x_i can be the same (i.e., all queens must be on different columns) and no two queens can be on the same diagonal.



BACKTRACKING

- N-Queens Problem**

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)	(1,8)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)	(2,8)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)	(3,8)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)	(4,8)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)	(5,8)
(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)	(6,8)
(7,1)	(7,2)	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)	(7,8)
(8,1)	(8,2)	(8,3)	(8,4)	(8,5)	(8,6)	(8,7)	(8,8)

N-Queens – Implementation Idea

- A recursive algorithm that places a queen in a column
- **Base case**
 - If there are no more columns to consider-
Solution Found
- **Recursive step**
 - If you successfully place a queen in the current column - Consider the next column
 - If you cannot place a queen in the current column- You need to backtrack



BACKTRACKING

Example of Backtrack solution to 4-Queens Problem

1			

(a)

1			
.	.	2	

(b)

1			
		2	
.	.	.	.

(c)

1			
			2
.	3		

(d)

1			
			2
	3		
.	.	.	.

(e)

	1		

(f)

	1		
.	.	.	2

(g)

	1		
			2
3			
.	.	4	

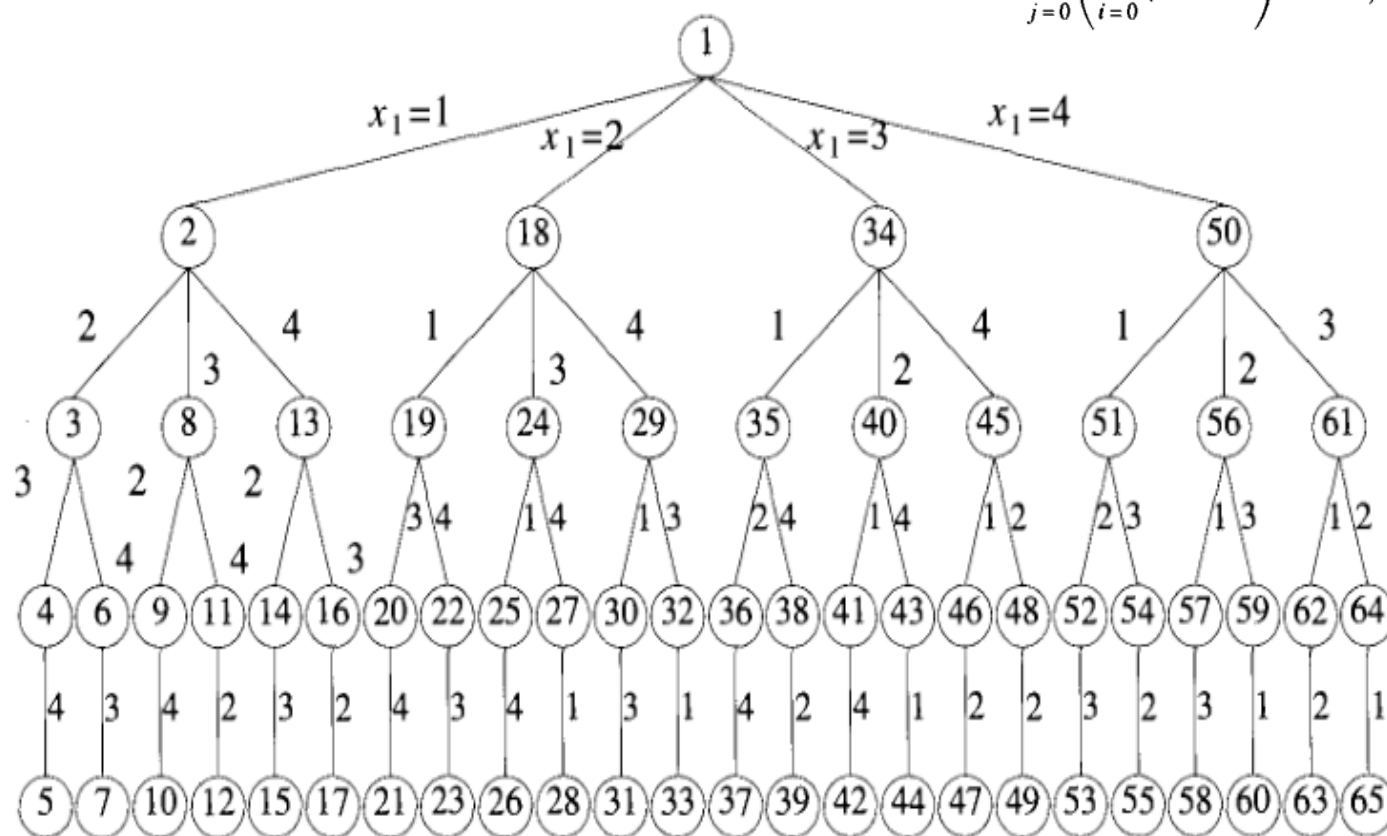
(h)



BACKTRACKING – State Space Tree

Tree organization of the 4-Queens Solution Space- Nodes are numbered in DFS order.

$$1 + \sum_{j=0}^7 \left(\prod_{i=0}^j (8 - i) \right) = 69,281.$$



BACKTRACKING

All Solutions to N-Queens Problem

Algorithm NQueens(k, n)
 // Using backtracking, this procedure prints all
 // possible placements of n queens on an $n \times n$
 // chessboard so that they are nonattacking.
 {
 for $i := 1$ to n do
 {
 if Place(k, i) then
 {
 $x[k] := i$;
 if ($k = n$) then write ($x[1 : n]$);
 else NQueens($k + 1, n$);
 }
 }
 }

Can a new queen be placed??

Algorithm Place(k, i)
 // Returns **true** if a queen can be placed in k th row and
 // i th column. Otherwise it returns **false**. $x[]$ is a
 // global array whose first $(k - 1)$ values have been set.
 // Abs(r) returns the absolute value of r .
 {
 for $j := 1$ to $k - 1$ do
 if ($(x[j] = i)$ // Two in the same column
 or ($\text{Abs}(x[j] - i) = \text{Abs}(j - k)$))
 // or in the same diagonal
 then return false;
 return true;
 }



BACKTRACKING

All Solutions to N-Queens Problem

Algorithm NQueens(k, n)

// Using backtracking, this procedure prints all
// possible placements of n queens on an $n \times n$
// chessboard so that they are nonattacking.

```
{  
    for  $i := 1$  to  $n$  do  
    {  
        if Place( $k, i$ ) then  
        {  
             $x[k] := i$ ;  
            if ( $k = n$ ) then write ( $x[1 : n]$ );  
            else NQueens( $k + 1, n$ );  
        }  
    }  
}
```



BACKTRACKING

Can a new queen be placed??

Algorithm Place(k, i)

// Returns **true** if a queen can be placed in k th row and
// i th column. Otherwise it returns **false**. $x[]$ is a
// global array whose first $(k - 1)$ values have been set.
// Abs(r) returns the absolute value of r .

```
{  
    for  $j := 1$  to  $k - 1$  do  
        if (( $x[j] = i$ ) // Two in the same column  
            or (Abs( $x[j] - i$ ) = Abs( $j - k$ )))  
            // or in the same diagonal  
            then return false;  
    return true;  
}
```



Home work

Analyse time Complexity??

